

09 — Generated Applications Standard

Objetivo. Definir, sin ambigüedad, qué constituye una *aplicación válida* generada por Puglit: la base determinista que toda app hereda (el Spine), los componentes obligatorios, los estándares de base de datos, API, seguridad, UI, performance, accesibilidad, testing y deployment, y el conjunto de *gates* automáticos que una build debe pasar antes de considerarse entregable. Este documento es el contrato. Cada afirmación está verificada contra el código real en `/Users/alvaroz/projects/2026/puglit ( spine/ , web/scripts/build-local.mjs , web/lib/app-builder.ts )`.

1. Generated App Philosophy

Puglit no genera *cualquier* aplicación: genera aplicaciones que comparten una **misma columna vertebral determinista** (el Spine) y un **mismo conjunto de invariantes** que el swarm de agentes nunca puede romper. La filosofía se resume en tres principios.

Half product, half determinism. Lo que es *commodity* en todo SaaS — autenticación, rate limiting, analytics, mailer, i18n, pool de Postgres — no se le pide al modelo. Vive pre-escrito en `spine/`, testado, y se ensambla en cada app. Los agentes sólo escriben lo *bespoke*: las tablas del dominio, los route handlers, las páginas. Esto sube el piso de calidad sin depender de un modelo mejor, y elimina la clase de bugs más cara (un JWT mal verificado, un pool mal configurado).

Real software, not a green compile. La definición de "válida" no es "compila". Una app válida **bootea, responde sin 5xx en sus páginas estáticas, carga su SQL contra un Postgres real, y pasa vitest + coverage**. El `RUNTIME GATE` de `build-local.mjs` existe precisamente porque un compilado verde miente: en el litmus test *Stayforge* (clon de Airbnb) el gate expuso **7 bugs reales** invisibles a `tsc`.

No hardcoding, no external deps. El estándar `DEV` (de `web/lib/app-builder.ts`, constante `RULES`) prohíbe paquetes npm fuera de React + `fetch` + el Spine — nada de `framer-motion`, `swr`, `axios`, `uuid`, `zod`, `jsonwebtoken`, `bcrypt` en rutas *bespoke*. Todo lo que el modelo necesita ya está en el Spine y se importa desde `@/lib/*`. El módulo `hardenRoute` (regla 6) elimina deterministamente cualquier import externo que se haya colado y lo reescribe al equivalente del Spine.

Pregunta	Respuesta del estándar
¿Qué escribe el modelo?	Tablas del dominio, route handlers, páginas, tests
¿Qué NO escribe nunca?	Auth, rate-limit, analytics, mailer, i18n, db pool
¿Qué prueba que "funciona"?	tsc → SQL load → vitest+coverage → boot+smoke (0 5xx)
¿Qué deps puede agregar?	Ninguna fuera de las del Spine
¿Quién es dueño del resultado?	El usuario (BYO GitHub/Vercel; Puglit no persiste tokens)

2. Mandatory Components

Toda app generada hereda, sin excepción, los siguientes componentes del Spine. Son **obligatorios**: un build que los pierda no es una app Puglit válida. Cada uno es genérico y agnóstico de dominio.

2.1 Auth — `spine/lib/auth.ts`

Sesión **stateless JWT** firmada con `JWT_SECRET` (HS256, vía `jose`). El token viaja como cookie `httpOnly` en web y como `Authorization: Bearer` en mobile; `getAuthUser(request)` acepta ambos transparentemente, de modo que **toda ruta es agnóstica web/mobile sin código extra**. El payload es estrictamente `{ userId, email, plan }` — sin acoplamiento de dominio.

```
// spine/lib/auth.ts (verbatim)
export interface JWTPayload { userId: number; email: string; plan: string }
export async function getAuthUser(request: NextRequest): Promise<JWTPayload | null> {
  const authHeader = request.headers.get("Authorization")
  if (authHeader?.startsWith("Bearer ")) return verifyJWT(authHeader.slice(7))
  const token = request.cookies.get("auth_token")?.value
  if (!token) return null
  return verifyJWT(token)
}
```

La cookie es `httpOnly`, `sameSite: "lax"`, `secure` en producción, `maxAge` 30 días (token web `30d`, mobile `365d`, refresh `365d`). El registro (`spine/app/api/auth/register/route.ts`) valida server-side: email por regex, `name` 2–60 chars con whitelist Unicode (bloquea HTML/JS injection via NFD strip), password ≥ 8 chars, hash `bcrypt` con cost 10. Campos de signup específicos del dominio se guardan en `profile` (JSONB) — **el Spine nunca hardcodea columnas de dominio**.

Sobre esto, `spine/lib/auth-guards.ts` centraliza el boilerplate de plan: `requireAuth` (401), `requirePaidPlan` (402, lleva al cliente a `/pricing`), `requirePlanAtLeast(minPlanId)`. Planes pagos y ranking se **derivan de `domain.config`** (`priceUsd > 0` = pago; rank = orden en `monetization.plans`) — sin nombres de plan hardcodeados.

2.2 Analytics — `spine/lib/analytics.ts` + `spine/lib/db.ts`

Tracking unificado: GA4 + Microsoft Clarity + **un funnel interno persistido**. El cliente (`analytics.ts`) expone `pageView`, `funnelStep`, `event`, `formError`, `setUser`; los IDs vienen de env (`NEXT_PUBLIC_*`). `funnelStep` mapea un embudo genérico de 8 pasos (`landing` → `signup_view` → `form_started` → ... → `purchased`) que el generador extiende por dominio.

Crucialmente, los eventos **se persisten, nunca se descartan**: `trackInternal` hace `POST /api/track` y `db.ts` los inserta en `analytics_events` (y `page_visits`). `trackEvent` es tolerante: si la tabla aún no existe en prod, hace no-op en vez de bloquear el request.

2.3 Mailer — `spine/lib/mailer.ts`

Primitiva transaccional mínima usada por el módulo de auth. Usa **Resend** cuando `RESEND_API_KEY` está seteada; si no, loggea a consola (modo dev). El `from` sale de `EMAIL_FROM` o `${config.identity.name}`

`<onboarding@resend.dev>` . El módulo completo de email-lifecycle (tracking, bounce poller, drip) sobrescribe esto con `sendTrackedEmail` cuando se habilita — auth sólo necesita una primitiva confiable.

2.4 Rate Limit — `spine/lib/rate-limit.ts` + `spine/middleware.ts`

Limitador **in-memory, sin dependencias, edge-compatible** (per-process; para exactitud multi-región se cambia el store por Redis). `middleware.ts` aplica límites por ruta, tuneados a propósito: auth generoso (CGNAT + typos no deben lockear), webhooks alto (reintentos del proveedor), pagos estricto.

Ruta	Límite	Ventana	Razón
<code>/api/auth/login</code>	10	60 s	abuso de credenciales
<code>/api/auth/register</code>	20	60 s	generoso (CGNAT + typos)
<code>/api/stripe/webhook</code>	100	60 s	reintentos del proveedor
<code>/api/track</code>	120	60 s	barato, alto volumen
<code>(default)</code>	60	60 s	resto de <code>/api/*</code>

El mismo middleware setea headers de seguridad (`X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy`) y una cookie de sesión de visitante para el funnel (saltea Bearer y páginas cacheables para no romper el cache del CDN). Un 429 devuelve `Retry-After` y `X-RateLimit-Remaining` . El generador extiende `RATE_LIMITS` con las rutas del dominio.

2.5 i18n — `spine/lib/i18n.tsx`

Contexto bilingüe mínimo (`LanguageProvider` + `useLanguage`). Las strings de *chrome* viven en `DICT` (en/es); el contenido de dominio (nombre, tagline, features) viene de `domain.config` resuelto vía `tr(Localized)` . El idioma por defecto es inglés si el producto lo soporta (estos demos se muestran en un sitio en inglés), si no el primer idioma del producto. La elección se persiste en `localStorage` . Agregar idiomas = extender `identity.languages` + `DICT` .

3. Folder Structure

Cada app generada es **Spine ensamblado + archivos bespoke que sobrescriben en colisión**. El Spine real es:

```

spine/
├── app/
│   ├── layout.tsx          # metadata SSR + LanguageProvider + fuente
│   ├── page.tsx            # landing (reemplazada por la app real)
│   ├── globals.css         # Tailwind v4
│   ├── login/ register/   # páginas de auth (chrome)
│   ├── app/                # dashboard autenticado (layout, [entity], account)
│   └── api/
│       ├── auth/           # register, login, me, logout, verify-email,
│       │                   # forgot/reset-password, resend-verification
│       ├── records/[entity]/ # CRUD genérico scoped al usuario
│       └── track/          # ingestión del funnel (persiste analytics_events)
├── components/             # AuthShell, AppSidebar, EntityManager, Mark
├── lib/
│   ├── auth.ts auth-guards.ts auth-emails.ts users.ts
│   ├── db.ts               # export const pool = new Pool()
│   ├── rate-limit.ts analytics.ts i18n.tsx mailer.ts records.ts
│   ├── middleware.ts       # rate limit + headers + cookie de sesión
│   ├── domain.config.ts    # identidad, módulos, entidades, monetización
│   ├── domain-types.ts     # DomainConfig, Localized
│   ├── next.config.ts      # standalone (no-Vercel), poweredByHeader:false
│   ├── tsconfig.json       # strict, alias @/* → ./*
│   └── package.json        # next 16, react 19, pg, jose, bcryptjs (sin más)

```

Durante el ensamblado (`build-local.mjs` → `assemble()`), un conjunto de superficies-template del Spine se **descarta** (`SPINE_DROP`) para que la app real las provea: `app/page.tsx` , `app/login` , `app/register` , `app/app/*` , los componentes de chrome y `domain.config.ts` . Sobre ese esqueleto se escriben los archivos bespoke (override en colisión), y se inyectan tres SQL: `sql/001_core.sql` , `002_auth.sql` , `003_records.sql` (del Spine) + el `sql/app.sql` y `sql/seed.sql` del dominio.

Una app **generada** típica (ej. el clon de status page por defecto) luce así tras el ensamblado:

```

.builds/puglit-<slug>/
├── app/
│   ├── page.tsx            # el producto real (overview)
│   ├── incidents/page.tsx  # superficies del dominio
│   ├── api/<recurso>/route.ts # route handlers bespoke
│   └── (spine app/api/auth, api/track intactos)
├── lib/<dominio>.ts        # lógica pura del dominio (lo que se testea)
├── lib/__tests__/*.test.ts # vitest del swarm/Queen
├── sql/{001_core,002_auth,003_records}.sql # Spine
├── sql/{app,seed}.sql      # dominio (reparados hasta cargar limpio)
├── domain.config.ts        # materializado del idea del usuario
├── next.config.ts vitest.config.ts .env.local # escritos por el builder
└── node_modules/          # hard-links del Spine (cp -al)

```

4. Naming Standards

Los nombres son parte del contrato porque otras capas (UI ↔ API ↔ SQL) deben coincidir exactamente. El generador los normaliza determinísticamente.

Elemento	Estándar	Origen / enforcement
Path de route handler	<code>app/api/<segments>/route.ts</code>	<code>canonicalRoutePath()</code> corrige nombres inválidos (p.ej. <code>/create.ts</code>)
Handlers HTTP	<code>export async function GET/POST/PUT/PATCH/DELETE</code>	<code>RULES</code> + <code>routeScore</code> (+50 si presentes)
Páginas	<code>app/<route>/page.tsx</code> ; home siempre <code>app/page.tsx</code>	blueprint exige <code>/</code> = el producto real
Columnas SQL	<code>snake_case</code> ; timestamp real <code>created_at</code>	<code>hardenRoute</code> regla 1 mapea columnas inventadas
Respuestas de lista	array desnudo , columnas <code>snake_case</code> exactas	<code>hardenRoute</code> regla 7 (<code>{data:rows}</code> prohibido)
Payload de auth	<code>u.userId</code> (number) — nunca <code>u.id</code>	<code>fixAuthUserFields()</code> reescribe <code>.id → .userId</code>
Dinero	<code>*_cents</code> (entero), nunca floats	playbook ARCHITECT + módulo rentals
Config	<code>domain.config.ts</code> (identity/modules/entities/monetization)	<code>domain-types.ts</code>

La regla de **shape de lista** merece énfasis: un `GET` que devuelve una colección **retorna el array desnudo** (`return NextResponse.json(rows)`), nunca envuelto en `{ matches }` o `{ data }`. Envolver es la causa #1 de `x.map is not a function` en la página — por eso `hardenRoute` lo desenvuelve determinísticamente.

5. Database Standards

Postgres crudo vía `pg`, sin ORM. El acceso es siempre `import { pool } from "@lib/db"` y `pool.query(text, params)`. No hay Prisma, Drizzle ni query builders.

SQL parametrizado, exclusivamente. `$1, $2, ...` — nunca interpolación de input del usuario. `routeScore` premia `\$\d` (+15) y penaliza fuerte (-40) cualquier `query(\ ...${...})`. El scanner de seguridad estático (`web/lib/swarm-checks.ts`) marca SQLi por interpolación.

Dinero en *integer cents*, nunca floats. El playbook y los prompts del arquitecto lo imponen como ley; el módulo de rentals lo materializa: `nightly_price_cents`, `cleaning_fee_cents`, `service_fee_cents`, `taxes_cents`, `total_cents` — todo entero. El pricing es determinista para que `search total === checkout total`.

EXCLUDE constraints para invariantes a nivel DB. El caso canónico es anti-double-booking, agregado automáticamente cuando se detecta un dominio de reservas (`web/lib/rentals-module.ts`):

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
ALTER TABLE <bookings> ADD CONSTRAINT no_double_booking
    EXCLUDE USING gist (listing_id WITH =, daterange(check_in, check_out, '[')') WITH &&;
```

Una reserva solapada dispara el código de error Postgres `23P01`, que la lógica del dominio traduce a un `409`. La correctitud vive en la base, no en la aplicación.

El pool está tuneado para el pooler. `spine/lib/db.ts` configura `max` (default 15, $\times N$ máquinas $\leq$ límite de la DB), `connectionTimeoutMillis` 8s, `allowExitOnIdle`, TLS por defecto (`POSTGRES_SSL=disable` sólo local), y un handler de error en el pool que loggea el código. En producción `ensureSchema()` se **saltea** — el DDL se envía como `scripts/sql/*.sql` para correr manualmente, porque el rol de la app no es dueño de la DB (y se conecta vía el **pooler Supervisor**, no el host directo IPv6-only).

Regla	Enforcement
Sólo <code>pg</code> crudo + <code>pool.query</code>	<code>SPINE_API</code> , <code>hardenRoute</code> regla 6
SQL parametrizado	<code>routeScore</code> , <code>swarm-checks.ts</code>
Integer cents	playbooks + rentals-module
<code>EXCLUDE</code> para overlaps	rentals-module (auto-inyectado)
<code>created_at TIMESTAMPTZ DEFAULT NOW()</code>	blueprint DDL template
Tablas referenciadas existen	<code>hardenRoute</code> regla 1 + <code>swarm-repair</code> (phantom-table)

6. API Standards

Toda API es un **App Router route handler** (Next 16). Los estándares "production-grade" (de `genRouteFile` en `web/lib/app-builder.ts`) son no-negociables:

Compleitud. Un handler por método necesario. Si el recurso es una *colección* que la UI lee y escribe (mensajes, comentarios, ítems), implementa **GET y POST**. Lecturas/escrituras sobre datos de otro usuario deben estar *scoped* (verificar que el caller es participante antes).

Status codes correctos. 200 read · 201 create · 400 bad input · 401 sin auth · 403 forbidden/no-participante · 404 missing · 409 conflict. JSON en todo path.

Validación de input antes de tocar la DB. Campos requeridos presentes, tipos primitivos correctos, rangos/longitudes sanas; input inválido → 400 con `{error:"..."}` (nunca dejar que llegue a SQL ni que tire 500). Números con `Number()` y rechazo de `NaN`.

Transaction safety. Toda mutación multi-statement que deba ser atómica corre en una transacción:

```
const c = await pool.connect()
try { await c.query("BEGIN"); /* ... */; await c.query("COMMIT") }
catch (e) { await c.query("ROLLBACK"); throw e }
finally { c.release() }
```

Resiliencia. Cuerpo del handler en `try/catch`; loggear y devolver `500 {error:"internal"}` ante fallo inesperado. Nunca filtrar objetos de error crudos al cliente.

Ejemplo del Spine (`api/auth/register`) que materializa el estándar: valida email/name/password, normaliza, hashlea con bcrypt, maneja el caso "email ya existe" como login (mejor UX que un muro 409), y envuelve todo en `try/catch` con 500 limpio.

Tras la generación, `hardenRoute` (un Reliability Engineer LLM) corrige bugs runtime que `tsc` no ve: conformidad de schema (toda columna referenciada existe — *bug #1*), scoping de participantes, exclusión del propio caller en feeds, correctitud de mutual-match, Haversine para "near me", remoción de deps externas, y shape de lista. La revisión es **determinista donde puede serlo**: el swipe→match se reemplaza por una ruta generada por código (`deterministicSwipeRoute`) cuando el shape calza, porque el LLM acierta el SQL pero invierte el binding de parámetros.

7. Security Standards

Vector	Defensa	Dónde
SQLi	SQL parametrizado obligatorio + scan estático	<code>swarm-checks.ts</code> , <code>routeScore</code>
Auth bypass	<code>getAuthUser</code> central; rutas per-user → 401 sin token	<code>auth.ts</code> , <code>auth-guards.ts</code>
Privilege escalation	<code>requirePaidPlan</code> / <code>requirePlanAtLeast</code> (402); plan derivado de config	<code>auth-guards.ts</code>
IDOR / cross-tenant	scoping de participante obligatorio (GET y POST)	<code>hardenRoute</code> reglas 2–3
Secrets en código	scan estático de secretos hardcodeados	<code>swarm-checks.ts</code>
Injection en inputs	validación server-side; whitelist Unicode en <code>name</code>	<code>api/auth/register</code>
Clickjacking / sniffing	<code>X-Frame-Options: DENY</code> , <code>nosniff</code> , <code>Referrer-Policy</code>	<code>middleware.ts</code>
Abuso / fuerza bruta	rate limit por ruta + 429 con <code>Retry-After</code>	<code>middleware.ts</code>
Cookie theft	<code>httpOnly</code> + <code>secure</code> (prod) + <code>sameSite: lax</code>	<code>auth.ts</code>
Deps maliciosas/innecesarias	cero deps externas fuera del Spine	<code>RULES</code> , <code>hardenRoute</code> regla 6
Leak de email entre usuarios	nunca exponer email; sólo alias/first name	prompt del arquitecto

La revisión adversarial (`web/lib/adversarial-review.ts`) agrega una capa de auditoría con tres lentes ortogonales — Auditor (lógica), Adversary (abuso/seguridad), Pragmatist (diseño) — con cross-examination determinista (≥2 lentes = confirmado) y veredicto **SHIP / SHIP-WITH-CAVEATS / BLOCK**. La regla de oro: **secretos siempre fail-closed**, ningún default inseguro que deje correr la app sin protección.

8. UI Standards

Stack. Next.js 16 (App Router) + Turbopack + React 19 + **Tailwind v4 únicamente** — no hay Shadcn/Radix ni librería de componentes; cada elemento se construye a mano con utility classes.

Boundaries RSC. Páginas cliente arrancan con `"use client"`. La normalización es determinista en dos lugares: `normalizeUseClient` en `app-builder.ts` (corrige `use client`; sin comillas → TS1434, lo agrega si la página usa hooks) y `ensureUseClient` en `build-local.mjs` (mismo invariante en el loop de reparación). Hooks y JSX viven dentro de **un** componente default-exported; todo hook se importa de `react`.

Reglas de navegación Next 16 (de `RULES`): `<Link href>` directo, **nunca** un `<a>` anidado dentro de `<Link>` (crash en runtime). Params dinámicos en páginas cliente con `useParams()` de `next/navigation` — nunca `router.query` (no existe en App Router).

Sin websockets, sin blob storage. Realtime/chat = `setInterval(fetch, 2500)` + `clearInterval` en unmount. Upload de imágenes = `<input type="file">` → `FileReader.readAsDataURL` → string `data:` URL guardado en una columna `TEXT`.

Design playbook (`VISUAL_SYSTEM`). Un *Art Director* LLM produce un brief de identidad por producto (dirección estética, arquitectura de layout, tokens de color con roles, escala tipográfica, recipes de componentes, motion, layout por pantalla), y cada agente de frontend lo sigue *verbatim*. La barra de calidad visual es un hard spec, no inspiración:

- **CONTRAST IS LAW:** todo nodo de texto con color explícito AA contra su propio fondo (el defecto #1 es texto blanco invisible sobre card blanca).
- Profundidad y jerarquía: escala de radios, shadows en capas, hairline borders.
- Los cuatro estados siempre: **LOADING** (skeletons `animate-pulse`), **EMPTY** (bloque amigable + CTA), **ERROR** (banner inline dismissible), **POPULATED**.
- Estados de interacción en cada elemento: hover, active, `focus-visible:ring`, disabled.
- **BANNED:** el look genérico de admin centrado y angosto, botones sin estilo, gris sobre gris, cards Bootstrap-ish.

Tras generar cada página corre un **QC loop reactivo** (`genPolishedPage`): genera → el QC critica contra el brief → refina.

9. Performance Standards

Estándar	Detalle
Turbopack	dev y build sobre Turbopack; <code>turbopack.root</code> pinneado a la app para evitar root inferido erróneo
<code>output: standalone</code>	imagen Docker chica (deshabilitado en Vercel, que maneja su propio tracing)
Listas paginadas	<code>?limit/?offset</code> con defaults sanos y <code>limit</code> capeado; <code>ORDER BY</code> una columna real
Columnas explícitas	seleccionar columnas, evitar <code>SELECT *</code> al shapear respuestas
Índices	el SQL del core indexa lo caliente (<code>idx_page_visits_created</code> , <code>idx_analytics_events_event</code> , ...)
Pool reusado	un único <code>pool</code> exportado, <code>max</code> capeado por máquina, <code>idleTimeoutMillis</code> 30s
Cache del CDN	la cookie de sesión se saltea en páginas públicas cacheables (<code>/</code> , <code>/blog</code> , <code>/_next</code> , <code>/static</code>)
<code>poweredByHeader: false</code>	sin header <code>X-Powered-By</code>
Fuente optimizada	<code>next/font/google</code> (Plus Jakarta Sans) con <code>variable</code>

El analytics del lado servidor es *fire-and-forget* tolerante: `trackEvent` nunca bloquea el request si la tabla no existe aún.

10. Accessibility Standards

La accesibilidad es parte de `VISUAL_SYSTEM` y se exige en cada pantalla:

- **Elementos semánticos:** `header` / `nav` / `main` / `section` / `button` (no `div` clickeables).
- `alt` en imágenes y `aria-label` en botones icon-only.
- **Foco visible:** `focus-visible:ring-2 ring-offset-2`.
- **El color nunca es la única señal** de estado.
- **Contraste AA** garantizado para cada par texto-sobre-superficie (la regla "CONTRAST IS LAW").
- **Tap targets $\geq 44\text{px}$** en mobile.
- Responsive mobile-first con refinamientos `sm:/md:/lg`.

La lente *Pragmatist* de la revisión adversarial vigila estos puntos de diseño además de la lógica.

11. Testing Standards

El testing es **vitest + coverage medida**, y no se confía en la afirmación del agente: se *mide*.

Config (`build-local.mjs` escribe `vitest.config.ts`). Entorno `node` , alias `@` → root de la app, coverage `v8` con reporters `text-summary` + `json-summary` , `include: ["lib/**/*.ts"]` , excluyendo tests y `__tests__` . Sólo se cubre la **lógica de dominio** en `lib/` — se saltean explícitamente los libs del Spine (`db` , `auth` , `mailer` , `rate-limit` , `analytics` , `i18n`).

Estilo (`genTests` + el prompt del Test Engineer). Arrange-Act-Assert: cada test estructurado Arrange → Act → Assert; nombrado por comportamiento — `describe("<unit>")`, `it("<expected behavior> when <condition>")`; **un comportamiento por test.** Se importan las funciones **reales** desde `@/lib/...` y se testean **sólo funciones puras** — nunca algo que toque DB/red. Se asertan **valores exactos** + casos borde (boundaries, input inválido, transiciones de estado).

Anti-patterns prohibidos (verbatim del prompt): `expect(true).toBe(true)`; snapshot-everything; asserts demasiado amplios; `test.skip`; borrar/debilitar una aserción para pasar; `await` faltante en async.

El Queen evidence gate. Tras compilar y cargar SQL, `runTests()` corre vitest+coverage y la **Abeja Reina** del equipo revisa la evidencia real: ¿tests verdes? ¿coverage ≥ bar (default **70%**, `PUGLIT_COVERAGE_BAR`)? Si no llega, la **rebota con feedback específico** ("coverage 41% < 70%, estas funciones sin cubrir: ...") y el equipo escribe más tests → se re-mide. Loop hasta cruzar el bar o agotar `PUGLIT_QA_ROUNDS` (default 2). El veredicto (✓ **APROBADO** / entregado con deuda de QA) se persiste en el status del export. Si no hay lógica de dominio testeable, se reporta coverage 0% honestamente.

12. Deployment Standards

El runtime gate de `build-local.mjs` es la línea de aceptación. No es "trust the claim": se ejecuta el pipeline completo localmente y se *sirve una app corriendo*. Las etapas, en orden:

1. **Generar** (los agentes: interview → spec → blueprint → routes/pages → 3 supervisiones).
2. **Pull** de los archivos generados desde Postgres.
3. **Ensamblar** Spine + bespoke en un proyecto runnable (con tres correcciones deterministas: `ensureSpineImports` prepone imports de Spine faltantes — `pool`, `getAuthUser`, `NextResponse` / `NextRequest` — que el modelo usa sin importar; `unescapeJsx` arregla TSX JSON- sobre-escapado; `ensureUseClient`).
4. **Cargar SQL** contra una DB fresca cada intento (spine → app → seed), **reparando DDL** que el modelo erró, hasta cargar limpio (hasta 5 rondas).
5. `tsc --noEmit` → se alimenta cada archivo con error + sus errores de vuelta al modelo → loop con anti-oscilación (guarda el mejor snapshot, para si 2 rondas no mejoran).
6. **QA gate** (vitest + coverage + Queen, §11).
7. `next dev` + **SMOKE TEST**: el `RUNTIME_GATE`. Bootea la app, camina el árbol `app/` recolectando rutas de páginas estáticas, las hitea (GET, sin side-effects) y **asierta cero 5xx**. `pages0k/pagesFail` se persisten en el status. Esto es el chequeo real de "¿realmente corre?".

Export BYO (Bring Your Own). El usuario es dueño del deploy. Si `GH_TOKEN` o `VERCEL_TOKEN` están en el env (pasados directo a `infra/deploy.sh`, **nunca persistidos**):

- **GitHub = salida universal** (cualquier tipo de app): crea el repo bajo *tu* cuenta y hace push. Siempre se corre.
- **Vercel = opcional**, sólo web apps: además da una URL pública live.

El status final (`status: done`, `compiles`, `githubUrl`, `vercelUrl`, `smoke`, métricas de QA) se escribe a `EXPORT_STATUS_FILE` para que el frontend lo muestre. Puglit no cobra nada por el deploy ni guarda tus tokens.

Resumen — definición de "app válida"

Una aplicación generada por Puglit es válida si, y sólo si:

1. Hereda intactos los **5 componentes obligatorios** del Spine (auth, analytics, mailer, rate-limit, i18n) y el pool de Postgres.
2. Usa **pg crudo + SQL parametrizado**, integer cents, e invariantes en la DB (**EXCLUDE**) donde aplica.
3. Sus APIs son route handlers con **status codes correctos, validación, transacciones y try/catch**, sin deps externas.
4. Su UI es Next 16 + Tailwind con los cuatro estados, contraste AA y accesibilidad.
5. **Compila** (**tsc**), su **SQL carga** contra Postgres real, **pasa vitest + coverage $\geq$ bar** (Queen), y **bootea sin 5xx** (runtime gate).
6. Es **exportable** a *tu* GitHub/Vercel — código que poseés, sin lock-in.